

Arrays in C

Representation, Addressing & Applications

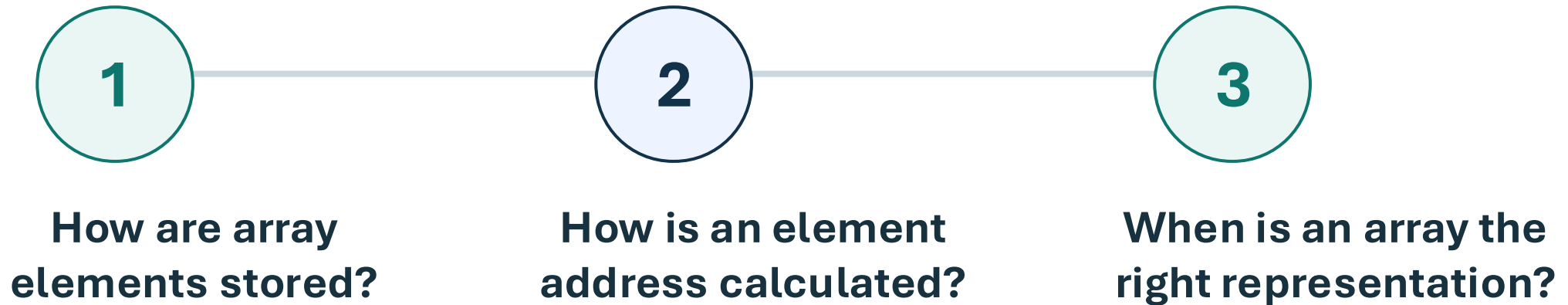
Unit I • Combined Lectures 5, 6 and 7

LECTURE 5 • 1D ARRAYS

LECTURE 6 • ADDRESSING

LECTURE 7 • APPLICATIONS

Three questions guide these lectures



- Trace one- and two-dimensional arrays in memory.
- Write C functions that receive arrays safely.
- Select ordered or sparse representations for the problem.

An array stores one type in contiguous memory

An array is a fixed-size, indexed sequence.

- Every element has the same data type.
- Elements occupy consecutive memory locations.
- The index identifies a position, not an address.
- Valid C indexes run from 0 to $n - 1$.

```
int marks[5] = {72, 81, 65, 90, 77};
```

index	0	1	2	3	4
	72	81	65	90	77
address	1000	1004	1008	1012	1016

Each int uses $w = 4$ bytes in this example.

Indexing gives every element a predictable address

$$\text{LOC}(A[i]) = \text{Base}(A) + (i - \text{LB}) \times w$$

Base(A)

address of the first logical element

LB

lower bound; LB = 0 for a normal C array

w

size of one element in bytes

WORKED EXAMPLE

A[6], Base = 500, w = 4
Find LOC(A[4])

$$500 + (4 - 0) \times 4 = 516$$

Declare and use one-dimensional arrays

DECLARATION + ACCESS

```
int score[5] = {82, 75, 91, 68, 88};
```

```
score[3] = 70;  
printf("%d", score[2]);    // 91
```

Read the declaration from right to left

- score is an array
- with 5 elements
- each element is int
- score[3] updates the fourth element

Bounds matter

score[5] is outside this array.

A loop traverses; an index updates

TRAVERSE ALL ELEMENTS

```
int sum = 0;
for (int i = 0; i < n; ++i) {
    sum += a[i];
}
```

UPDATE ONE ELEMENT

```
a[2] = a[2] + 5;

// direct access to one element
```

Operation	Cost
Access $a[i]$	$O(1)$
Update $a[i]$	$O(1)$
Traverse n values	$O(n)$

The loop is linear because every element contributes once.

Insertion and deletion require shifting elements

Insert 30 at index 2

0	1	2	3	4
10	20	40	50	—

shift right →

0	1	2	3	4
10	20	30	40	50

Delete the value at index 2

0	1	2	3	4
10	20	30	40	50

← shift left

0	1	2	3	4
10	20	40	50	—

Worst-case insertion or
deletion: $O(n)$

A two-dimensional array is still one linear block

Logical view

11	12	13
21	22	23

rows × columns = 2 × 3



Physical order in C

11	12	13	21	22	23
----	----	----	----	----	----

row 0 first

then row 1

C DECLARATION

```
int m[2][3] = {  
    {11, 12, 13},  
    {21, 22, 23}  
};
```

C stores 2D arrays in row-major order

$$\text{LOC}(A[i][j]) = \text{Base} + w \times [(i - L_r) \times N + (j - L_c)]$$

N = number of columns

One complete row contributes N elements to the offset.

Storage sequence

a00	a01	a02	a10	a11	a12
-----	-----	-----	-----	-----	-----

skip i rows

then move j columns

offset = i × N + j

Use the row-major formula to compute any address

Given: `int A[3][4]`, Base = 1000, w = 4 bytes

A00	A01	A02	A03
A10	A11	A12	A13
A20	A21	A22	A23

Target: `A[2][1]`

1 N = 4 columns

2 $\text{offset} = 2 \times 4 + 1 = 9$

3 $\text{byte displacement} = 9 \times 4 = 36$

4 $\text{LOC}(A[2][1]) = 1000 + 36 = 1036$

Column-major order changes only the stride

$$\text{LOC}(A[i][j]) = \text{Base} + w \times [(j - L_c) \times M + (i - L_r)]$$

M = number of rows

Row-major sequence

a00	a01	a02	a10	a11	a12
-----	-----	-----	-----	-----	-----

Column-major sequence

a00	a10	a01	a11	a02	a12
-----	-----	-----	-----	-----	-----

C uses row-major order. Column-major order is useful for comparison and address-analysis questions.

Three dimensions extend the same address logic

For $A[D1][D2][D3]$ in row-major order:

$$\text{offset} = ((i - L1) \times D2 + (j - L2)) \times D3 + (k - L3)$$

PLANES

$i \times D2 \times D3$

ROWS

$j \times D3$

COLUMNS

k

Example: $A[2][3][4]$, target $A[1][2][3] \rightarrow \text{offset} =$
 $((1 \times 3) + 2) \times 4 + 3 = 23$

Pass an array by sending its first element's address

ARRAY AS A PARAMETER

```
void printArray(const int a[], int n) {  
    for (int i = 0; i < n; ++i)  
        printf("%d ", a[i]);  
}  
  
printArray(score, 5);
```

What the function receives

- `a` refers to the first element.
- The array length is not carried automatically.
- Pass `n` explicitly to keep the loop in bounds.
- `const` prevents accidental element changes.

`a[i]` means `*(a + i)`

A 2D parameter needs its column size

FIXED COLUMN COUNT

```
void printMatrix(int rows, int a[][4]) {  
    for (int i = 0; i < rows; ++i)  
        for (int j = 0; j < 4; ++j)  
            printf("%d ", a[i][j]);  
}
```

Why is 4 required?

The compiler needs the row stride to calculate:

$$\text{offset} = i \times 4 + j$$

C99 variable-length alternative

```
void f(int r, int c, int  
a[r][c]);
```

Arrays simplify counting, lookup, and tabulation

1

Frequency counting

`count[value]++` records how often a value occurs.

2

Searching and lookup

Indexes provide direct access; ordered arrays support binary search.

3

Tables and matrices

Rows and columns model marks, images, grids, and numeric data.

4

Building other structures

Stacks, queues, heaps, and strings can use arrays.

Question: Is the size known?

Ordered arrays speed search but slow updates

Ordered list

0	1	2	3	4	5
12	18	25	31	44	—

Insert 27 →

0	1	2	3	4	5
12	18	25	27	31	44

Invariant: after every operation, $a[0] \leq a[1] \leq \dots \leq a[n-1]$

Binary search

$O(\log n)$

Find insert position

$O(\log n)$

Shift elements

$O(n)$

Sparse matrices should store only non-zero values

Dense view

0	0	7	0	0
0	0	0	0	9
5	0	0	0	0
0	0	0	8	0

4 × 5 matrix; 4 non-zero values



Triplet representation

row	col	value
4	5	4
0	2	7
1	4	9
2	0	5
3	3	8

Store (row, column, value) for each non-zero entry.

Choose the representation that matches the data

Need constant-time indexed access



1D array

Model a rectangular C table



2D row-major array

Search often; update rarely



ordered array

Most matrix entries are zero



sparse triplet

EXIT TICKET `int B[5][6], Base = 800, w = 4. Find LOC(B[3][4]).`

Show the formula, element offset, byte displacement, and final address.